
VVU EARTH TECH

Developer Specification — Technical Outreach Edition

FOR OUTREACH DISTRIBUTION, SCOPING & SALES FRAMEWORK STRATEGIES

Target Audience: Developers, Integrators, Technical Partners

CLASSIFICATION: Outreach Distribution — Scoping & Sales Framework Strategy

Content Scope: Technical, Operational, and Strategic — Excludes Legal & Financial

Publication Date: 2026-07-28 | Version 1.0

Venture Vision Ubuntu (VVU) — EARTH TECH Division

Table of Contents

- 1. Architecture Overview
- 2. Epistemic Primitives & Type System
- 3. Dependency Injection Framework
- 4. Canonicalization & Hashing (RFC 8785 JCS + SHA-256)
- 5. Merkle Mountain Range (MMR)
- 6. Trust Runtime API
- 7. Gate Pipeline Architecture
- 8. Risk Score Engine
- 9. Circuit Breaker State Machines
- 10. 72-Hour Resilience Matrix
- 11. Feature Gate & Hard Failure Codes
- 12. VETPS Proof Standard
- 13. 5-Pass Evidence Compiler
- 14. Integration Guide

1. Architecture Overview

The **VVU EARTH TECH** platform is a deterministic evidence runtime for infrastructure monitoring, municipal water network management, and autonomous decision-making. It combines an **Epistemic DAG Runtime** (with four primitives: Fact, Proof, Policy, Projection), a **Trust Runtime** (5-state AIR safety pipeline), and a **72-Hour Resilience Matrix** to deliver verifiable, replay-deterministic, append-only evidence for every decision the system makes.

Two-Layer Architecture

- **Layer 1 — Epistemic Runtime (Horizontal Infrastructure):** Deterministic evidence runtime, MMR, canonicalization, replay engine, acceptance pipeline. Open-source under Apache-2.0. NO product-specific logic (Golden Rule enforced by AST scanner).
- **Layer 2 — Product Applications (Vertical):** Municipal water monitoring (HBK), infrastructure management, decision dashboards. Commercial tier, builds on Layer 1 via adapters.

Directory Structure

Path	Layer	Access	Content
open-source/air-kernel/	Layer 1	Apache-2.0	Core runtime, MMR, replay, acceptance pipeline
open-source/epistemic-runtime/	Layer 1	Apache-2.0	Fact/Proof/Policy/Projection primitives
open-source/safe-krypte-basic/	Layer 1	Apache-2.0	Basic cryptographic operations
commercial/feature-gate.ts	Layer 2	Commercial	requireFeature() with HF-006 enforcement
shared/license/	Cross-layer	Shared	License schema, validator
shared/vetps/	Cross-layer	Shared	VETPS proof standard schemas
shared/verifiers/	Cross-layer	Shared	VerifierRegistry interfaces

2. Epistemic Primitives & Type System

Primitive	Type Signature	Required Fields	Invariant
Fact	interface Fact { id, timestamp, body, sourceHash, mmrIndex }	id, body, sourceHash	Append-only; SHA-256(body) immutable
Proof	interface Proof { id, factId, signerId, algorithm, signature }	factId, signerId, signature	Ed25519/RSA-PSS-SHA256/ECDSA P-384
Policy	interface Policy { id, version, rules, effectiveFrom, effectiveTo }	version, rules	Bi-temporal; replay-deterministic
Projection	interface Projection { id, sourceFacts, computedAt, value }	sourceFacts, value	Recomputable from Fact Log alone

3. Dependency Injection Framework

The AIR Kernel uses constructor-based dependency injection for all external dependencies. This enables testing with mock implementations, runtime swapping of storage/signing backends, and deterministic replay with injected clocks and UUID generators.

Injectable	Interface	Default Implementation	Testing Use
Clock	now(): number	Date.now()	Injected fixed timestamp for replay
UUID	generate(): string	crypto.randomUUID()	Deterministic UUIDs for test replay
Entropy	bytes(n): Uint8Array	crypto.getRandomValues()	Fixed entropy for deterministic tests
Signer	sign(data, algorithm): Signature	Ed25519 (default)	Mock signer for non-production
EvidenceStore	append(fact): MMRIndex	S3 Object Lock COMPLIANCE	In-memory store for tests
ProjectionRegistry	register/projection()	HashMap-based	Test projections with fixed outputs
PolicyEngine	evaluate(facts, policy): Decision	Rule-based evaluator	Deterministic policy for replay
MMR	append/verify/proof()	In-memory MMR	Pre-seeded MMR for replay
Canonicalizer	canonicalize(obj): string	RFC 8785 JCS	Fixed canonicalization for hashing
ReplayEngine	replay(fromTimestamp): ReplayResult	Full replay from Fact Log	Deterministic replay with injected deps
AcceptancePipeline	accept(raw): AcceptedFact	5-Pass compiler	Mock pipeline for unit tests
VerifierRegistry	register/verify()	Ed25519 + RSA + ECDSA	Add custom verifiers per deployment

4. Canonicalization & Hashing

RFC 8785 JCS (JSON Canonicalization Scheme) is the sole canonicalization method. All objects are serialized to a deterministic JSON string before hashing. This guarantees that the same logical object always produces the same hash, regardless of key order, whitespace, or encoding differences.

Hashing: SHA-256 Only

- **SHA-256** is the sole hashing algorithm across the entire system.
- No SHA-1, no MD5, no Blake2, no custom hash functions.
- All Fact bodies, MMR nodes, evidence bundles, and state bundles use SHA-256.
- Canonicalization → SHA-256 is applied as: `hash(SHA-256(JCS(object)))`

Signing Algorithms

Algorithm	Use Case	Key Type	Implementation
Ed25519	Default signing for Facts and Proofs	Elliptic curve (Curve25519)	Native crypto + AWS KMS
RSA-PSS-SHA256	High-assurance institutional signing	RSA 2048/4096	AWS KMS + IAM Federation
ECDSA P-384	Governance anchor signing (GovernanceAnchor.sol)	Elliptic curve (P-384)	AWS KMS + OIDC

5. Merkle Mountain Range (MMR)

The Epistemic Runtime uses a **real Merkle Mountain Range** (not a binary Merkle tree). MMR provides efficient append-only proofs with logarithmic verification, supporting both inclusion proofs and consistency proofs without requiring the full tree.

- **Append-Only:** New Facts are appended to the MMR without modifying existing peaks.
- **Peak Hashing:** MMR peaks are hashed together to produce the current root hash.
- **Inclusion Proofs:** Any Fact can be proven to be in the MMR with $O(\log n)$ path.
- **Consistency Proofs:** Prove that the current MMR root is a valid extension of any previous root.
- **Bagging:** Peak bagging algorithm compresses multiple peaks into a single root for compact verification.

6. Trust Runtime API

The Trust Runtime provides the 5-state AIR safety pipeline with 7 gates, a composite risk score engine, and a hysteresis circuit breaker. It operates in two modes: **observe** (Phase 1-3: compute but do not enforce) and **enforce** (Phase 4: compute and halt execution on TRIPPED/ESCALATED).

API Endpoint

```
/api/trust-runtime – GET returns current AIR state, risk score, gate metrics
```

GateMetrics Interface

```
interface GateMetrics { exposure: [0,1]; failures: [0,1]; entropy: [0,1]; velocity:  
[0,1]; acceleration: [0,1]; intentAge: [0,1]; drift: [0,1]; convergencePenalty:  
[0,1]; }
```

■ **ALL GateMetrics values MUST be normalized to [0,1] before reaching the risk score engine. Unnormalized values trigger AIRUnnormalizedMetricError — this is a bounded-state invariant, not a cosmetic issue.**

7. Gate Pipeline Architecture

The gate pipeline orchestrates: Temporal Validity → Convergence → Accumulation → Velocity → Acceleration → State Drift → Risk Score → Circuit Breaker. It is a pure function — the caller owns persisting state.

Gate	Function	Input	Output	Enforcement
Gate 0	Temporal Validity	Intent + now	normalizedAge or expiry	Phase 4: reject expired intents
Gate A	Convergence	Contraction ratio	convergencePenalty [0,1]	Penalty grows if diverging
Gate B	Accumulation	Exposure samples	normalized exposure [0,1]	Ceiling breach → trip
Gate C	Velocity	Exposure history	localVelocity + trip flag	Phase 4: halt if exceeded
Gate D	Acceleration	Velocity windows	deviation + trip flag	Phase 4: halt if exceeded
Gate E	State Drift	currentState vs snapshotState	distance + normalized [0,1]	Phase 4: halt if exceeded
Composite	Risk Score	All GateMetrics	score [0, ~1.3]	EWMA-smoothed, delta-tracked
Final	Circuit Breaker	score + deltaScore	AIRState transition	Hysteresis with escalation

8. Risk Score Engine

The composite risk score is a weighted sum of all gate outputs. It is an **early-warning indicator**, NOT a proven-stable control quantity (formerly mislabeled "Lyapunov" — corrected in implementation).

Weight	Value	Metric Source
exposure	0.25	Gate B — Accumulation
failures	0.20	Decaying counter — adapter failures
entropy	0.10	Epistemic Runtime — belief-state contradiction
velocity	0.10	Gate C — first derivative
acceleration	0.10	Gate D — second derivative
intentAge ²	0.10	Gate 0 — intent age squared (normalized)
drift	0.15	Gate E — state drift distance
drift × intentAge	0.10	Cross term — drift × age interaction
convergencePenalty	0.20	Gate A — divergence penalty

■ **Weights are UNDERIVED defaults (P0). Phase 3 observation required before Phase 4 enforcement tuning.**

9. Circuit Breaker State Machines

AIR Safety Pipeline (5-State)

NORMAL → WARNING → TRIPPED → RECOVERY → NORMAL. TRIPPED → ESCALATED after 3 trips within 1-hour window. RECOVERY has minimum 5-minute hold to prevent flapping.

Municipal Infrastructure (3-State)

NORMAL → DEGRADED → FAIL-CLOSED. DEGRADED allows reduced throughput. FAIL-CLOSED rejects all new submissions. This breaker is architecturally independent from the AIR safety pipeline.

10. 72-Hour Resilience Matrix

Pillar	Component	Mechanism	Guarantee
NATS/HLC	Hybrid Logical Clock	Durable queue + (wall_time, logical, node_id)	Partition tolerance
Fail-Closed Circuit Breaker	5-state hysteresis	NORMAL→WARNING→TRIPPED→RECOVERY→ESCALATED	Oscillation prevention
Hydro-Gateway	IoT Sensor Bridge	Acoustic leak detection → Epistemic Runtime	Telemetry ingestion
CSB/WAL	Cryptographic State Bundle	SHA-256 + CRC32c append-only log	Evidence integrity
Policy Time Travel	Bi-temporal evaluation	Evaluate policies at any historical timestamp	Governance audit

11. Feature Gate & Hard Failure Codes

The feature gate system enforces tier boundaries with hard failure codes. Any attempt to access a feature above the current tier throws HF-006.

Code	Description	Penalty	Gate Location
HF-001	Mock boolean / No TEE Verifier	0.31 penalty	Evidence Compiler Pass 2
HF-002	No ZK Prover	Critical	GovernanceAnchor.sol
HF-006	Feature BLOCKED by invalid license	Hard block	requireFeature()
HF-007	Tenant Boundary Violation	Critical	namespace = SHA-256(tenant_public_key)

Feature Gate Implementation

```
requireFeature(featureName): decorator that checks CACHED_VALIDATION.isValid and CACHED_VALIDATION.features.includes(featureName). Throws HF-006 on violation.
```

12. VETPS Proof Standard

VETPS (VVU Earth Tech Proof Standard) bridges HBK $\leftrightarrow$ AIR. It defines the schema for proof packages that connect municipal infrastructure evidence to the Epistemic Runtime. VETPS proofs are structured, signed, and verifiable by any third party.

- **Schema:** VETPS proof packages contain: evidence hash, signer identity, algorithm, signature, timestamp, and MMR inclusion proof.
- **Verification:** Any party can verify a VETPS proof by: (1) canonicalizing the evidence, (2) computing SHA-256, (3) verifying the signature, (4) checking the MMR inclusion proof.
- **HBK Adapter:** The `hbk-adapter.ts` implements SHA-256 hash verification for municipal telemetry, converting Hydro-Gateway sensor data into VETPS-compliant evidence.

13. 5-Pass Evidence Compiler

Pas s	Name	Operation	Output	Hard Failure Gate
1	Ingest	Receive raw telemetry/sensor data	RawObservation	—
2	Canonicalize	RFC 8785 JCS + SHA-256	CanonicalFact	HF-001 (bad signature → quarantine)
3	Redact	Policy-based field redaction	RedactedFact	—
4	Infer	Policy evaluation + projection computation	InferredFact + Decision	HF-005 (contradiction → TRIP)
5	CodeGen	Generate Proof + MMR append	AcceptedFact + Proof + MMRIndex	HF-002 (bad ZK → reject)

14. Integration Guide

Prerequisites

- Node.js 18+ or Bun runtime
- Next.js 16 with App Router
- TypeScript 5 strict mode
- Prisma ORM (SQLite client)
- Tailwind CSS 4 + shadcn/ui

API Endpoints

Endpoint	Method	Returns	Layer
/api/trust-runtime	GET	AIR state, risk score, gate metrics	Layer 1
/api/resilience	GET	Resilience manager status	Layer 1
/api/resilience/circuit-breaker	GET	3-state breaker status	Layer 1
/api/kernel	GET	Kernel runtime status	Layer 1
/api/kernel/verify	POST	Verification result	Layer 1
/api/vvu-strategy	GET	7-Track strategy data	Layer 2
/api/validation-suite	GET	VAL-001 phases + milestones	Layer 2
/api/metrics	GET	Performance metrics	Layer 1
/api/policies	GET/ POST	Policy CRUD	Layer 1

Strategy & Outreach

7-Track Resource Acquisition & Partnership Strategy:

- Track A — Municipal Partnerships:** Active outreach to Cape Town and progressive municipalities for pilot deployment and validation partnerships. *Status: Active Outreach*
- Track B — University Research Partnerships:** Engagement with South African and international universities for collaborative research, validation observation, and academic publication. *Status: Strategy*
- Track C — Industry Integration:** Partnership development with water infrastructure, IoT, and municipal technology companies for integration, supply chain, and co-development. *Status: Strategy*
- Track D — Public Funding:** Application to national and provincial technology innovation funds, water sector grants, and digital infrastructure programmes. *Status: Strategy*
- Track E — Private Investment:** Structured engagement with impact investors, technology venture capital, and ESG-aligned funds. *Status: Strategy*
- Track F — Sponsorship & Equipment:** Outreach to hardware manufacturers, cloud providers, and technology sponsors for equipment, infrastructure, and in-kind support. *Status: Strategy*
- Track G — Community & Open Source:** Building developer community, open-source contributors, and civic technology networks for grassroots validation and adoption. *Status: Strategy*

Execution Principle: VVU EARTH TECH advances through multiple independent pathways in parallel. Every activity is subject to formal governance review. Progress is measured by verified engineering deliverables, not announcements. Where a pathway encounters obstacles, alternative pathways continue without delay. This parallel execution model ensures continuous forward progress regardless of individual pathway outcomes.